

Assessment

Overview

Build a production-quality, feature-rich system demonstrating:

- Solid mobile UX (Expo + RN), close fidelity to a modern food app menu UX.
- Clean, scalable REST APIs (Express + Sequelize + MySQL).
- Secure auth (JWT with refresh, rotation).
- Robust data modeling, validation, error handling, and documentation.
- Strong testing and delivery hygiene (Jest, CI, seeds, lint, type-safety).

Scope note: This is a “developer test” build, but every decision should be production-leaning (security, performance, maintainability). Prefer pragmatic completeness over breadth.

Tech Stack Requirements

Frontend

- **Framework:** Expo (React Native CLI is also acceptable)
- **State Management:** Zustand (Redux is also acceptable)
- **Data Fetching:** React/Tanstack Query + Axios
- **Navigation:** Expo Router (React Navigation is also acceptable)

Backend

- **Framework:** Express.js
- **Database:** MySQL with Sequelize ORM

General

- **Language:** TypeScript (mandatory throughout the stack)
- **Testing:** Jest for unit testing (Bonus: Maestro for e2e testing)

Project Requirements

Frontend Development

Menu Interface

The menu interface should be identical to the Deliveroo menu interface. Match behavior & layout patterns (not brand assets). For any other screen, you can follow any UI design guidelines and conventions.

[Link to UI](#) (Open this link in a mobile view)

- Restaurant information section
- Category navigation with sticky headers
- All sections for food items
- Food item cards including images, descriptions, and pricing
- Interactive food item modals/popups
- Smooth scrolling across categories
- Mobile-first, responsive design

Authentication

- Email/password signup & login
- Include at least one social login (Google or Apple). Should demonstrate a full round trip to the backend.
- Protect authenticated routes using route guards
- Persistent session with access token and rotating refresh token

Technical Features

- Set up Fonts (IBM Plex Sans), App icons, bundle IDs, and Splash screens appropriately
- Set up dark mode/light mode using a global state
- Integrate with Backend APIs for data (Load restaurant/menu data from the database along with authentication)
- TanStack Query for caching, retries with exponential backoff

- Offline-aware UX hints (loading, error, retry states) include network checks and proper warning and error handling

Backend Development

Note: Only API endpoints are required for the following. No admin dashboard or UI for backend features is needed. Evaluation will be based on API design, documentation, and code quality.

User Management

- Secure registration and login flows
- Custom JWT authentication system with access and refresh tokens
- Middleware for protecting authenticated routes

Restaurant Management

- Database schema for restaurant entities
- CRUD API endpoints
- Seed sample data using **faker.js**

Menu Management

- Schema for categories, dishes, and ratings
- CRUD APIs with pagination support
- Data seeding using **faker.js**

Order Management

- Schema to manage order data (items, quantities, pricing, timestamps)
- APIs for placing, updating, and viewing orders
- Seed at least **10,000** orders for performance testing

Reporting

Implement API endpoints for key reports:

- Total sales by day/week/month
- Top-selling items (by quantity or revenue)
- Average order value by time period

Support advanced querying with:

- Filtering by order status
- Sorting by priority
- Date range filtering
- Pagination

Data Validation and Error Handling

- Use **zod** or **yup** for request validation
- Provide clear, meaningful error messages
- Gracefully handle unexpected or invalid operations

Documentation and Deliverables

Required

- Complete source code in GitHub repositories with:
 - Organized folder structure
 - Clear commit history and branches
- Fully functional React Native app with the specified features
- Fully functional Express app with the specified features.
- Documentation for:
 - [README.md](#) with setup and run instructions
 - API documentation for the backend (Swagger/Postman)
 - Migration scripts (if applicable)

Bonus (Optional)

- Localization scaffold (i18n) with at least 1 extra locale stub
- Order placement from the mobile app
- Push notifications for order placement
- Shopping cart with live updates

Evaluation Criteria

- Code quality, structure, and maintainability
 - Completeness of core features
 - Responsiveness and attention to detail
 - Performance and scalability considerations
 - Security and authentication best practices
 - Quality and completeness of documentation
-

Timeline

You have **7 days** to complete this assessment. Focus on delivering core functionality with solid architecture. You are encouraged to implement bonus features only if time permits and without compromising on quality.

Note on AI Usage

AI tools may be used to assist with development. You must **mention all the AI tools** that you have used in the documentation and **fully understand and be able to explain all architectural and implementation decisions** during the code review.